Passenger Assignment Model in a Public Transportation Network

Michel Bierlaire Evangelos Paschalidis

July 30, 2026

1 Introduction

This document describes the modeling assumptions, mathematical formulation, and inference components implemented in the software. It is intended as technical documentation for users and developers of the package.

The objective of this software is to infer an aggregate representation of public transportation demand from operational data. The demand is characterized by a flow for each origin–destination (OD) pair and each desired departure time interval.

The software follows four main steps:

1. Develop a detailed network representation of the public transportation system.
2. Formulate an assignment model that maps an OD matrix to passenger flows on the network.
3. Define measurement equations linking model outputs to observed data, yielding either a likelihood or an explicitly weighted linear objective.
4. Detect topology-driven structural-zero OD cells before estimation.
5. Estimate demand using ML, MAP, fixed-routing linear and block-coordinate methods, or Bayesian variational inference.

2 Modeling Elements

The following sections document the mathematical objects and modeling assumptions used internally by the software.

2.1 Spatial Representation

Let $\mathcal{S}$ denote the set of stops in the area of interest. Each stop can act as an origin, a destination, or a transfer location.

Definition 1 (OD pair)**.** *An origin–destination pair is a tuple*

$$q = (s_o, s_d) \in \mathcal{Q} = \mathcal{S} \times \mathcal{S}.$$

2.2 Temporal Representation

The analysis period is partitioned into a set $\mathcal{T}$ of departure time intervals. Each interval represents a desired departure time window.

Definition 2 (Departure time interval)**.** *Each time bin* $\mathtt{t} \in \mathcal{T}$ *corresponds to a closed interval*

$$[\mathtt{a}_\mathtt{t}, \mathtt{b}_\mathtt{t}] \subset \mathbb{R}, \qquad \mathtt{a}_\mathtt{t} < \mathtt{b}_\mathtt{t},$$

where $\mathtt{a}_\mathtt{t}$ *and* $\mathtt{b}_\mathtt{t}$ *denote the start and end time (in minutes) of the desired departure interval.*

For each departure interval $\mathtt{t} \in \mathcal{T}$, the desired departure window is the interval $[\mathtt{a}_\mathtt{t}, \mathtt{b}_\mathtt{t}]$.

Definition 3 (OD demand)**.** *For each OD pair* $\mathtt{q} \in \mathcal{Q}$ *and departure time interval* $\mathtt{t} \in \mathcal{T}$*, let*

$$\mathtt{f}_\mathtt{q}^\mathtt{t} \geqslant 0$$

denote the number of passengers wishing to depart during interval $\mathtt{t}$ *from origin* $\mathtt{s}_\mathtt{o}$ *to destination* $\mathtt{s}_\mathtt{d}$*.*

The full demand vector is denoted by

$$\mathbf{f} = \{\mathtt{f}_\mathtt{q}^\mathtt{t} : \mathtt{q} \in \mathcal{Q},\ \mathtt{t} \in \mathcal{T}\}.$$

2.3 Baseline demand and Bayesian deviation parameterization

The model assumes that the analyst can provide an *a-priori* demand matrix $\mathbf{f}^{(0)} = \{\mathtt{f}_\mathtt{q}^{\mathtt{t},(0)}\}$, aligned with the OD/time-bin indexing of the scenario. Rather than placing a prior directly on the demand levels $\mathbf{f}$, we parameterize demand in terms of *log-deviations* from $\mathbf{f}^{(0)}$. This improves numerical stability, facilitates weakly informative priors, and makes the role of prior information explicit.

Modeling choice (smoothly bounded log-deviation). For each OD/time-bin cell, introduce an unconstrained latent variable $\widetilde{z}_\mathtt{q}^\mathtt{t} \in \mathbb{R}$. To protect the exponentiation and the downstream assignment from extreme values while retaining a smooth gradient, define the effective log-deviation as

$$z_\mathtt{q}^\mathtt{t} = z_{\max} \tanh\left(\frac{\widetilde{z}_\mathtt{q}^\mathtt{t}}{z_{\max}}\right), \qquad z_{\max} > 0.$$

The demand used by the assignment is then

$$\mathtt{f}_\mathtt{q}^\mathtt{t} = \mathtt{f}_\mathtt{q}^{\mathtt{t},(0)} \exp(z_\mathtt{q}^\mathtt{t}).$$

This transformation guarantees $\mathtt{f}_\mathtt{q}^\mathtt{t} \geqslant 0$ whenever $\mathtt{f}_\mathtt{q}^{\mathtt{t},(0)} \geqslant 0$ and preserves the baseline structure up to multiplicative adjustments. It also implies

$$|z_\mathtt{q}^\mathtt{t}| \leqslant z_{\max}, \qquad \exp(-z_{\max}) \leqslant \frac{\mathtt{f}_\mathtt{q}^\mathtt{t}}{\mathtt{f}_\mathtt{q}^{\mathtt{t},(0)}} \leqslant \exp(z_{\max})$$

for positive baseline cells. The default $z_{\max} = 6$ therefore permits demand multipliers ranging approximately from 1/403 to 403. In the software configuration, this smooth bound is exposed under the historical name `z_clip`; despite that name, the implementation uses a smooth hyperbolic-tangent transformation rather than hard clipping.

Zero baseline cells. If $\mathtt{f}_\mathtt{q}^{\mathtt{t},(0)} = 0$, then $\mathtt{f}_\mathtt{q}^\mathtt{t} = 0$ for all values of $\widetilde{z}_\mathtt{q}^\mathtt{t}$. Therefore, OD cells that are zero in the baseline matrix remain structurally zero during inference. In order to allow strictly positive deviations from zero, a small but non zero seed value must be provided: $\mathtt{f}_\mathtt{q}^{\mathtt{t},(0)} = \varepsilon > 0$.

Prior scale. The raw unconstrained variables, rather than the bounded effective deviations, are assumed independent and normally distributed:

$$\widetilde{z}_q^t \sim \mathcal{N}(0, \sigma_z^2).$$

The parameter $\sigma_z > 0$ controls the strength of regularization. Small values of σ_z constrain demand to remain close to the baseline, while large values allow the effective deviations to approach their smooth bounds. In the current version of the software, σ_z is a fixed hyperparameter.

The distinction between $\widetilde{z}_q^t$ and z_q^t is important: the Gaussian density is evaluated at the raw variable $\widetilde{z}_q^t$, whereas the assignment and all reported demand samples use the transformed value z_q^t.

Separation of responsibilities. The assignment procedure is treated as a deterministic mapping

$$(\mathbf{f}, \theta) \longmapsto \mathbf{x}(\mathbf{f}; \theta)$$

and remains agnostic to priors and baseline information. The construction of $\mathbf{f}$ from $\mathbf{f}^{(0)}$ and $\widetilde{\boldsymbol{z}}$ is performed in the *Bayesian layer* (implemented using NumPyro and stochastic variational inference), which: (i) reads the baseline $\mathbf{f}^{(0)}$ provided by the user, (ii) draws the raw deviations $\widetilde{\boldsymbol{z}}$ from their prior distribution, (iii) applies the smooth bound to obtain $\boldsymbol{z}$, and (iv) computes $\mathbf{f}$ using the expression above before invoking the assignment. This design avoids introducing additional indexing conventions and keeps the assignment module independent of the inference engine.

Desired departure interval and admissible boarding window. For each OD pair $q \in Q$ and departure time interval $t \in \mathcal{T}$, we associate the desired departure *interval* $[a_t, b_t]$ and an admissible schedule-deviation window of half-width $\Delta_{\max}^{\text{access}} > 0$ (user-defined, e.g. 15 minutes).

Admissible access links. An access link is included in the graph *only if* the departure time τ is not too far from the desired interval, namely

$$\tau \in \left[a_t - \Delta_{\max}^{\text{access}},\ b_t + \Delta_{\max}^{\text{access}}\right],$$

where τ denote the actual departure time (in minutes) of a boarding at the origin stop. As described later, the boarding is modeled by a link, that is included if the distance from τ to the interval $[a_t, b_t]$ does not exceed $\Delta_{\max}^{\text{access}}$.

Schedule-deviation penalty. When the access link is included, its generalized cost is defined by a (possibly asymmetric) piecewise-linear penalty with a *flat* (zero-penalty) region inside the interval:

$$c_{\text{access}}(\tau; t) = \beta_{\text{early}}\ \max\big(0,\ a_t - \tau\big) + \beta_{\text{late}}\ \max\big(0,\ \tau - b_t\big),$$

where $\beta_{\text{early}} \geqslant 0$ and $\beta_{\text{late}} \geqslant 0$ are user-defined coefficients. Therefore,

$$c_{\text{access}}(\tau; t) = 0 \quad \text{whenever} \quad \tau \in [a_t, b_t],$$

and penalties apply only to departures strictly earlier than a_t or strictly later than b_t.

Note that the thresholds $\Delta_{\max}^{\text{access}}$ and $\Delta_{\max}^{\text{transfer}}$ are fixed configuration parameters; the assignment is differentiable w.r.t. costs and OD flows given a fixed graph.

Implementation note (departure interval as OD/time-bin data). Although centroid-in nodes are not time-tagged in the graph, the desired departure *interval* $[\mathtt{a_t}, \mathtt{b_t}]$ remains an attribute of the OD/time-bin pair. In computations, the access-link penalty is evaluated by combining the interval bounds $(\mathtt{a_t}, \mathtt{b_t})$ with the clock time τ of the *departure event node* reached by that access link. This yields a zero penalty for departures within $[\mathtt{a_t}, \mathtt{b_t}]$ and an early/late penalty based on the nearest bound otherwise, without introducing backward-in-time links in the acyclic time-expanded network.

2.4 Network Representation

The public transportation system is represented as a directed time-expanded graph. All timetable times (arrival/departure event times) are expressed in minutes on the same time axis as the departure-time intervals.

A key modeling choice is (i) the separation of each stop centroid into two distinct nodes (an origin *centroid-in* and a destination *centroid-out*), and (ii) the split of each timetable event into an *arrival* and a *departure* node. This second split prevents ill-defined instantaneous moves (e.g., transferring or boarding at the exact same time as arrival) and ensures that all links respect a strict chronological (or lexicographic) order, so that the resulting time-expanded network is acyclic.

Definition 4 (Nodes)**.** *The node set consists of three families of nodes:*

- ***Centroid-in nodes (non time-tagged, indexed by departure interval):*** *for each stop* $\mathtt{s} \in \mathcal{S}$ *and for each departure interval* $\mathtt{t} \in \mathcal{T}$*, a node*

$$(\mathtt{s}, \mathtt{t})^{\text{in}}$$

  used as a demand-injection handle for passengers departing from stop $\mathtt{s}$ *with desired departure interval* $\mathtt{t}$*. These nodes do not correspond to a physical time instant; they are placed* before all event nodes *in the topological order (conceptually at* $-\infty$*). The interval index* $\mathtt{t}$ *is used only to (i) select admissible boarding events through the access-window rule, and (ii) compute schedule-deviation penalties.*

- ***Centroid-out nodes (non time-tagged):*** *for each stop* $\mathtt{s} \in \mathcal{S}$*, a node* $\mathtt{s}^{\text{out}}$ *representing the point where passengers arriving at stop* $\mathtt{s}$ *exit the network. These nodes are not associated with a physical time; they are placed* after all event nodes *in the topological order (conceptually at* $+\infty$*) so that the global graph remains acyclic.*

- ***Timetable event nodes (trip-specific, split into arrival and departure):*** *for each stop* $\mathtt{s}$*, each scheduled arrival time* τ^{arr} *and departure time* τ^{dep} *associated with a given vehicle run (trip), we create two nodes*

$$(\mathtt{s}, \tau^{\text{arr}})^{\text{arr}} \qquad \textit{and} \qquad (\mathtt{s}, \tau^{\text{dep}})^{\text{dep}}.$$

  These nodes are trip-specific*: if two trips serve the same stop at the same clock time, they still correspond to different event nodes.*

Definition 5 (Lines and event-to-line mapping)**.** *Let* $\mathcal{L}$ *denote the set of public transport lines. Each timetable event node (arrival or departure) is associated with exactly one operated line, captured by the mapping*

$$\lambda : \mathcal{V}^{\text{event}} \to \mathcal{L}, \qquad \nu \mapsto \lambda(\nu),$$

where $\mathcal{V}^{\text{event}}$ *is the set of timetable event nodes. In practice,* $\lambda(\nu)$ *is derived from the timetable: event nodes created from a given trip inherit the line identifier of that trip. Therefore, for a given trip at stop* $\mathtt{s}$*, the corresponding arrival and departure nodes share the same line label.*

Centroid-in nodes are not time-tagged: they are placed before all timetable event nodes (conceptually at $-\infty$) so that access links can connect to departure events that occur either before or after the desired departure window without violating acyclicity. Centroid-out nodes are not time-tagged and are placed after all event nodes (conceptually at $+\infty$), preserving a global topological order.

Remark. The separation of centroid-in and centroid-out nodes is essential for the validity of the forward dynamic-programming loading algorithm. Without this separation, egress links could break the acyclic structure and violate the topological order implied by time.

Definition 6 (Links)**.** *The link set $\mathcal{A}$ contains the following link types:*

1. ***Access links (time-windowed):*** *for an OD pair $\mathtt{q} = (\mathtt{s_o}, \mathtt{s_d})$ and departure interval $\mathtt{t}$, access links connect the origin centroid-in node $(\mathtt{s_o}, \mathtt{t})^{\text{in}}$ to* selected departure *event nodes at the origin stop,*

$$(\mathtt{s_o}, \mathtt{t})^{\text{in}} \to (\mathtt{s_o}, \tau)^{\text{dep}},$$

only for departure times τ satisfying

$$\tau \in \left[\mathtt{a_t} - \Delta^{\text{access}}_{\max},\ \mathtt{b_t} + \Delta^{\text{access}}_{\max}\right].$$

2. ***Ride links:*** *for each trip, ride links connect a departure event at the current stop to an arrival event at the next stop,*

$$(\mathtt{s_i}, \tau^{\text{dep}}_{\mathtt{i}})^{\text{dep}} \to (\mathtt{s_{i+1}}, \tau^{\text{arr}}_{\mathtt{i+1}})^{\text{arr}}.$$

3. ***Dwell/continue links (within-trip at a stop):*** *for each trip and each stop event, a dwell link connects the arrival and departure nodes at the same stop,*

$$(\mathtt{s}, \tau^{\text{arr}})^{\text{arr}} \to (\mathtt{s}, \tau^{\text{dep}})^{\text{dep}}.$$

In the implementation, strictly positive dwell time is enforced by regularizing timetable records where arrival equals departure (using a small minimum dwell time).

4. ***Transfer links (inter-line, time-windowed):*** *transfer links connect an* arrival *event node to a later* departure *event node at the same stop and represent waiting and switching* between different lines*. A transfer link*

$$(\mathtt{s}, \tau_1)^{\text{arr}} \to (\mathtt{s}, \tau_2)^{\text{dep}}, \qquad \textit{with} \qquad \tau_2 > \tau_1$$

is included only if

   (a) the two event nodes belong to different lines*, and*

   (b) the waiting time satisfies $\tau_2 - \tau_1 \leqslant \Delta^{\text{transfer}}_{\max}$.

   No transfer links are created between events of the same line. In particular, transfer links are never created for $\tau_2 = \tau_1$.

5. ***Egress links (global graph, destination-gated by masking):*** *the global graph contains egress links from every arrival event node at stop $\mathtt{s}$ to the centroid-out node $\mathtt{s}^{\text{out}}$:*

$$(\mathtt{s}, \tau)^{\text{arr}} \to \mathtt{s}^{\text{out}}.$$

   For a given OD pair $\mathtt{q} = (\mathtt{s_o}, \mathtt{s_d})$, only the destination centroid-out node $\mathtt{s}^{\text{out}}_{\mathtt{d}}$ is enabled as a terminal node; all other egress options are disabled by a destination-dependent mask.

Each link $\ell \in \mathcal{A}$ is associated with:

- a generalized cost c_ℓ,
- a capacity u_ℓ for ride links (stored in the data structure, not yet used in costs).

Assumption 1 (Acyclic time-expanded network)**.** *All links in the time-expanded network are consistent with a strict topological order. All* event-to-event *links strictly increase clock time. In addition, centroid-in nodes* $(s,t)^{\text{in}}$ *are placed* before *all event nodes (conceptually at* $-\infty$*), and centroid-out nodes* s^{out} *are placed* after *all event nodes (conceptually at* $+\infty$*). With this convention, access and egress links always respect the global ordering.*

With the event split, feasible movements follow the pattern

$$(s,t)^{\text{in}} \to (s,\tau)^{\text{dep}} \to (s',\tau')^{\text{arr}} \to (s',\tau'')^{\text{dep}} \to \cdots \to (s_d,\bar{\tau})^{\text{arr}} \to s_d^{\text{out}}.$$

Transfer links satisfy $\tau_2 > \tau_1$ *and dwell links are enforced to be strictly forward in time through a minimum dwell regularization. Therefore the directed graph is acyclic and admits a topological ordering, which allows value functions and flows to be computed using dynamic programming.*

2.5 Generalized Cost

Each passenger seeks to minimize a generalized cost that includes:

- in-vehicle travel time (ride links),
- waiting and transfer time (transfer links) and dwell time at stops (dwell/continue links),
- transfer disutility through coefficient β_{transfer},
- early/late departure penalty relative to the desired departure *interval* (access links; zero penalty within $[a_t, b_t]$).

For a path p, the generalized cost is

$$C(p) = \sum_{\ell \in p} c_\ell.$$

2.5.1 Link-type-specific cost formulation

The generalized cost of each link depends on its type. All coefficients appearing below are fixed user-defined parameters.

Ride links. For a ride link ℓ representing the movement of a vehicle between two stops, connecting a departure event to a subsequent arrival event,

$$(s_i, \tau_i^{\text{dep}})^{\text{dep}} \to (s_{i+1}, \tau_{i+1}^{\text{arr}})^{\text{arr}},$$

the generalized cost is equal to the in-vehicle travel time:

$$c_\ell = \tau_{i+1}^{\text{arr}} - \tau_i^{\text{dep}}.$$

Dwell/continue links. For a dwell/continue link at a stop,

$$(s,\tau^{\text{arr}})^{\text{arr}} \to (s,\tau^{\text{dep}})^{\text{dep}},$$

the generalized cost is the dwell time at the stop:

$$c_\ell = \tau^{\text{dep}} - \tau^{\text{arr}}.$$

Transfer links (inter-line, bounded waiting). Consider a transfer link $\ell : (\mathtt{s}, \tau_1)^{\mathrm{arr}} \to (\mathtt{s}, \tau_2)^{\mathrm{dep}}$ with $\tau_2 > \tau_1$. Such a link is included only if (i) the two events correspond to *different lines* and (ii) the waiting time $\tau_2 - \tau_1$ does not exceed the user-defined threshold $\Delta_{\max}^{\mathrm{transfer}}$. When included, its generalized cost is

$$\mathtt{c}_\ell = \beta_{\mathrm{transfer}} \, (\tau_2 - \tau_1),$$

where $\beta_{\mathrm{transfer}} > 0$ is a user-defined coefficient.

Access links (bounded schedule deviation with a flat region). Consider an access link from the centroid-in node $(\mathtt{s_o}, \mathtt{t})^{\mathrm{in}}$ to an origin *departure* event node $(\mathtt{s_o}, \tau)^{\mathrm{dep}}$. The link is included only if

$$\tau \in \left[\mathtt{a_t} - \Delta_{\max}^{\mathrm{access}}, \ \mathtt{b_t} + \Delta_{\max}^{\mathrm{access}}\right].$$

When included, its generalized cost is

$$\mathtt{c}_\ell = \beta_{\mathrm{early}} \max(0, \mathtt{a_t} - \tau) + \beta_{\mathrm{late}} \max(0, \tau - \mathtt{b_t}),$$

with user-defined coefficients $\beta_{\mathrm{early}} \geqslant 0$ and $\beta_{\mathrm{late}} \geqslant 0$.

Egress links to centroids. For an egress link $\ell : (\mathtt{s}, \tau)^{\mathrm{arr}} \to \mathtt{s}^{\mathrm{out}}$, we set $\mathtt{c}_\ell = 0$.

Interpretation of cost coefficients. All time quantities in the model are measured in minutes. Travelers may perceive these minutes differently depending on context. The coefficients $\beta_{\mathrm{transfer}}$, β_{early} and β_{late} act as perception weights converting physical time into generalized cost. These coefficients are fixed and specified by the user; they are not estimated within the inference procedure. Only the OD demand and optionally the dispersion parameter θ are estimated.

3 Assignment Model

3.1 Input

The input is the OD vector $\mathbf{f}$ (scenario order), which is injected through centroid-in nodes.

3.2 Assignment Procedure

The classical assignment procedure based on deterministic shortest paths is not differentiable with respect to link costs and is therefore not well suited for gradient-based calibration methods. We adopt a differentiable assignment model inspired by the logit-based approach of Dial (1971), adapted to the acyclic time-expanded public transportation network.

3.3 Differentiable Dial-style assignment

Let $\mathbf{f}$ denote the OD demand. For each OD pair $\mathtt{q}$ and departure interval $\mathtt{t}$, passengers choose routes according to a logit model defined on the time-expanded network.

3.3.1 Capacity constraints (current status)

Although ride links are associated with nominal capacities $\mathtt{u}_\ell$ in the data structure, the current implementation does not incorporate capacity-dependent penalties into generalized costs. All assignment computations are therefore performed without congestion or overload effects.

Introducing capacity-aware penalties while preserving differentiability (e.g., using smooth overload functions such as softplus penalties) is a natural future extension, but it is not part of the current version of the software.

3.3.2 Logit routing model

Passengers are assumed to choose routes according to a logit model with dispersion parameter $\theta > 0$. For any path p,

$$\Pr(p) \propto \exp\left(-\frac{C(p)}{\theta}\right), \qquad C(p) = \sum_{\ell \in p} c_\ell.$$

When θ is small, flows concentrate on lowest-cost paths. When θ is large, flows spread across multiple attractive alternatives.

Computational and behavioral motivation. The logit formulation admits an efficient dynamic-programming implementation on the acyclic time-expanded network. It is differentiable with respect to generalized costs and, consequently, with respect to OD demand. This property is essential when the assignment model is embedded in gradient-based calibration and variational inference.

3.3.3 Differentiable dynamic programming formulation

Fix an OD pair $q = (s_o, s_d)$ and a departure interval t. The *terminal* node is the destination centroid-out node s_d^{out}, and the value function is initialized as

$$V(s_d^{\text{out}}) = 0.$$

For any other node i, let $\mathcal{A}_q^+(i)$ denote the set of outgoing links that are enabled for the destination group. We define

$$V(i) = -\theta \log \sum_{\ell=(i\to j)\in\mathcal{A}_q^+(i)} \exp\left(-\frac{c_\ell + V(j)}{\theta}\right).$$

This recursion is evaluated in reverse topological order using a numerically stable `logsumexp` formulation.

The probability that a passenger located at node i chooses link $\ell = (i \to j)$ is

$$P_\ell(i) = \frac{\exp\left(-\frac{c_\ell + V(j)}{\theta}\right)}{\sum_{(i\to k)\in\mathcal{A}_q^+(i)} \exp\left(-\frac{c_{ik} + V(k)}{\theta}\right)}.$$

3.3.4 Flow propagation

Fix an OD pair $q = (s_o, s_d)$ and a departure time interval $t \in \mathcal{T}$. The corresponding demand f_q^t is injected at the origin centroid-in node $(s_o, t)^{\text{in}}$.

Let y_i denote the flow reaching node i during the forward pass. Initialization is

$$y_{(s_o,t)^{\text{in}}} = f_q^t, \qquad y_i = 0 \quad \text{for all } i \neq (s_o, t)^{\text{in}}.$$

Given transition probabilities $P_\ell(i)$ on outgoing links $\ell = (i \to j)$, link flows are

$$x_\ell = y_i \, P_\ell(i), \qquad \ell = (i \to j),$$

and node flows are propagated in topological order by

$$y_j \leftarrow y_j + x_\ell, \qquad \ell = (i \to j).$$

Destination gating. The global graph contains egress links for all stops, but for a given OD pair $q = (s_o, s_d)$ the model enables only egress links leading to s_d^{out} via a destination-dependent mask.

Batched evaluation. For computational efficiency, value functions and flow propagation are computed in batches for OD groups sharing the same destination (and, depending on the implementation, the same departure interval), enabling vectorized evaluation with JAX.

3.3.5 Separation of routing preparation and demand loading

For a destination group g, let d_g denote its destination, let $\mathcal{A}_g^+$ denote its enabled links after destination gating, and let $\mathbf{f}_g$ collect the OD/time-bin demand injected for that group. The backward value recursion and the resulting transition probabilities depend on $(\mathcal{G}, \mathbf{c}, d_g, \mathcal{A}_g^+, \theta)$, but not on $\mathbf{f}_g$. The assignment can therefore be written as two distinct operations:

$$\mathcal{R}_g(\mathcal{G}, \mathbf{c}, d_g, \mathcal{A}_g^+, \theta) = (\mathbf{V}_g, \mathbf{P}_g),$$

followed by

$$\mathbf{x}_g = \mathcal{L}_g(\mathcal{G}, \mathbf{P}_g, \mathcal{A}_g^+, \mathbf{f}_g), \qquad \mathbf{x} = \sum_g \mathbf{x}_g,$$

where $\mathcal{R}_g$ is routing preparation and $\mathcal{L}_g$ is the forward flow-loading operation. For fixed probabilities, $\mathcal{L}_g$ is linear in the injected demand. This separation is exact: it does not approximate the route-choice model or alter its probabilities.

Precomputation for fixed dispersion. When θ is fixed during inference, and generalized costs do not depend on the current demand, the routing state $(\mathbf{V}_g, \mathbf{P}_g)$ is invariant across all likelihood evaluations. The software consequently prepares the effective destination masks and link probabilities once, stores them in an immutable JAX-compatible cache, and evaluates only $\mathcal{L}_g$ as the OD parameters change. Reverse-mode differentiation then traverses the demand-loading and measurement computations, but not the invariant value-function and route-probability calculations.

This optimization is valid under the current uncongested assignment model, because link costs are fixed. It must not be applied when θ is estimated, or if a future model makes generalized costs depend on passenger flows. In those cases, routing is recomputed inside every assignment evaluation. The ML, MAP, and variational-inference pipelines select the cached path automatically for fixed θ and retain the dynamic path for estimated θ.

Cache consistency. A prepared cache is associated with the exact graph instance, base link costs, destination-group ordering, and original destination masks from which it was constructed. These quantities are checked before cached loading is used. A change to any routing-sensitive input invalidates the cache. Empty destination sets and compact OD layouts are handled explicitly. The cached scan accumulates only total link flow and does not materialize per-destination link-flow arrays when they have not been requested.

Computational consequences. Let G be the number of active destination groups, N the number of graph nodes, and A the number of links. Routing preparation requires one backward pass and probability construction per group, while each subsequent cached evaluation requires only the forward loading passes. The principal incremental cache arrays scale as $O(GA)$ for probabilities and masks. Compact OD grouping can reduce both G and this storage when frozen-zero demand removes complete destination groups.

Table 1 reports illustrative CPU measurements from the package examples. Timings are warm medians using 32-bit arithmetic and are machine-dependent; numerical equivalence is tested separately and these values are not used as test thresholds.

Example and layout	G	Cache (MiB)	Forward	Value–gradient	Speedup
Simple example, compact	7	0.20	9.44→2.75 ms	12.78→6.21 ms	2.06×
Geneva, compact	18	3.43	0.449→0.109 s	0.537→0.180 s	2.98×
Geneva, full	62	11.83	1.552→0.383 s	1.788→0.591 s	3.03×

Table 1: Dynamic-to-cached fixed-routing measurements. The final column reports the likelihood value-and-gradient speedup; forward speedups range from 3.43× to 4.12×.

3.3.6 Direct fixed-routing measurement operator

Fixed routing permits a second, more aggressive optimization when inference needs measurement predictions but not link-level outputs. For fixed transition probabilities, every forward DAG update consists only of additions and multiplication by constants. It is therefore linear in the injected OD demand. Measurement aggregation is also linear. Their composition can be represented by a matrix $\boldsymbol{A}$:

$$\boldsymbol{\lambda} = \boldsymbol{A}\mathbf{f}, \qquad \boldsymbol{\mu} = \rho\boldsymbol{A}\mathbf{f}.$$

This identity is exact, rather than a local linearization.

For a compact layout containing free cells and positive frozen cells, the implemented representation separates variable columns from constants:

$$\boldsymbol{\mu} = \rho\left(\boldsymbol{A}_{\text{free}}\mathbf{f}_{\text{free}} + \mathbf{b}_{\text{fixed}}\right), \qquad \mathbf{b}_{\text{fixed}} = \boldsymbol{A}_{\text{fixed}}\mathbf{f}_{\text{fixed}}.$$

Frozen-zero cells have neither columns nor an offset contribution. Operator construction proceeds destination group by destination group, but does not construct unit-demand link-flow columns. Instead it combines loading and measurement aggregation in one reverse dynamic program. For destination group g, let $\mathsf{H}_{\mathsf{g}}(\mathsf{i}, \mathsf{m})$ be the expected accumulated contribution to measurement m of one passenger starting at node i. In reverse topological order,

$$\mathsf{H}_{\mathsf{g}}(\mathsf{i}, \mathsf{m}) = \sum_{\ell \in \delta^+(\mathsf{i})} \mathsf{P}_{\mathsf{g}\ell}\left[\mathbf{1}\{\ell \in \mathcal{A}_{\mathsf{m}}\} + \mathsf{H}_{\mathsf{g}}(\mathsf{h}(\ell), \mathsf{m})\right],$$

where $\mathsf{h}(\ell)$ is the head of link ℓ. Operator columns are obtained by gathering H_{g} at the OD origin nodes. Consequently, no `chunk_size`-by-$|\mathcal{A}|$ link-flow array is constructed or transferred to the host.

Only measurements mapped to links enabled for a destination group are carried by that group's recursion. The local measurement dimension is padded to a single fixed maximum, and every OD output chunk is padded to the configured chunk size. The builder explicitly lowers and compiles this fixed-shape kernel once, then reuses the executable for every group and chunk, including the final partial chunk. For BCOO output, nonzero data and row/column indices are appended directly from returned local contributions; a dense operator is never allocated and subsequently converted.

Construction profiling. Each chunk reports its ordinal number, fixed shape, completed columns, elapsed time, host peak memory, kernel dispatch, device synchronization, and NumPy transfer. Final metrics record the explicit compilation count and time, number of chunks, construction and assembly time, logical dense bytes, stored bytes, nonzero count, density, host peak memory, and device peak memory when the backend exposes it. Measurement aggregation has no separately dispatched phase because it is fused into the reverse loading kernel; it is therefore not double-counted as a separate device operation.

Validity and fallback. The direct operator is valid only while dispersion, routing probabilities, link costs, graph structure, OD ordering, and the measurement mapping remain fixed. It is invalid when dispersion is estimated, costs depend on demand, capacity or congestion

feeds back into routing, or graph, layout, or mapping indices change. The normal fixed-routing loader remains the reference and fallback implementation and is always required when link-level outputs are requested. The estimated-dispersion path never constructs or uses the direct operator.

Persistent cache and provenance. Operators can be saved as compressed NumPy archives and reused by a fresh process. The cache key includes the assignment identity; content digests of the graph and all loading arrays; the complete measurement mapping; OD-layout and compact-layout fingerprints, including frozen indices and values; fixed dispersion; representation; numeric type; package version; and cache-schema version. Loaded dimensions and array shapes are also validated. Cache writes use a temporary file followed by an atomic replacement. A missing, corrupt, outdated, or provenance-mismatched artifact is rejected and rebuilt safely. The cache directory may be supplied on the estimation request or through `PUBLIC_TRANSPORTATION_OPERATOR_CACHE_DIR`. Dense and JAX BCOO representations remain available as explicit overrides.

Activation policy. The modes are `off`, `auto`, `dense`, and `bcoo`. Explicit modes are preserved. In automatic mode, a valid BCOO cache is reused immediately. Without a valid cache, construction is selected only when the expected number of evaluations multiplied by the measured per-evaluation saving exceeds the expected construction time. Missing construction evidence or a nonpositive saving keeps the ordinary fixed-routing loader active. Estimated dispersion always disables the operator irrespective of this policy.

TPG structural measurement. For the TPG case, the operator has shape $3{,}690 \times 15{,}772$, or 58,198,680 logical entries. Only 158,219 entries are nonzero, giving a density of 0.271860%. The logical dense float32 size is 222.01 MiB, whereas the direct BCOO representation occupies 1.811 MiB. This observed sparsity, rather than an a-priori assumption, motivates BCOO as the automatic representation.

Chunk	Calls	Compile (s)	Build (s)	Process peak (GiB)	Warm value–gradient (s)
128	184	0.237	116.587	4.212	0.001115
256	124	0.238	79.264	4.220	0.001133
512	113	0.241	71.999	4.222	0.001075
1024	113	0.238	72.781	4.224	0.001164
2048	113	0.244	72.544	4.210	0.001104

Table 2: TPG direct-BCOO construction by fixed OD chunk size. The 2048 case was the largest tested safe value. At 512 and above every destination group fits in one call, so larger chunks do not reduce the 113 calls.

These measurements were rerun on the EPFL Scitas Jed server with Python 3.14.5, JAX 0.11.0, and NumPyro 0.21.0. At chunk size 1024, device synchronization accounted for 71.316 s of the 72.781 s construction; NumPy transfer took only 0.0065 s and compilation occurred exactly once. Relative to the 3.3866 s Jed reference evaluation, the measured construction break-even is approximately 21.5 evaluations. Fresh processes validated and loaded the persistent cache in 0.0203–0.0210 s. The chunk-1024 construction process peaked at 4.224 GiB RSS; the complete sequential benchmark batch peaked at 6.16 GiB.

Across three distinct demand vectors, the maximum absolute differences were 0.00390625 for the objective, 0.001953125 for the likelihood, and 2.861×10^{-6} for any gradient coordinate, consistent with float32 rounding. Tests also cover raw measurement predictions, frozen-zero cells, positive frozen offsets, cache round trips, corrupt-cache rebuilds, and provenance mismatch rejection.

Iterations	Evaluations	Loader (s)	Cached BCOO (s)	Speedup	Objective difference
1	4	14.814	0.950	15.6×	0.0000
5	8	34.514	7.127	4.8×	0.0039
20	24	104.566	17.413	6.0×	0.0078

Table 3: Fresh-process TPG optimizer timings from the unchanged profiling harness on Jed. Compilation is reported separately by that harness. Warm cached BCOO value-and-gradient time was 0.001007–0.001276 s, versus 3.378727–3.657539 s for the fixed-routing loader.

3.3.7 Scalability limits for very large OD layouts

The direct operator is not suitable when its dimensions or its construction state exceed the available memory. In the TPG full-network case there are 628,164 free OD cells and 426,436 measurements. A dense float32 operator would require approximately 998 GiB. More importantly, the reverse operator construction described above would require a destination-local state proportional to the number of graph nodes times the number of reachable measurements. Measurements from a sampled destination cover approximately 213,000 observations, implying about 344 GiB of value state for one destination. The explicit builder must therefore not be used for this layout.

Six isolated full-network destinations spanning 9, 137, 276, 465, 770, and 1,181 free cells were measured under a 24 GiB process ceiling. Warm forward loading was 5.98–6.08 s and ordinary value–gradient evaluation was 12.07–12.38 s; peak resident memory was 16.02–18.16 GiB. The near-constant runtime over a 131-fold change in OD count shows that graph traversal, not demand injection, dominates. These measurements establish that destination streaming alone is not a viable large-scale solver: it repeats expensive graph traversals and does not exploit the linear structure available when routing is fixed. The large-scale mode should instead expose the fixed-routing assignment as a sparse linear operator and solve the resulting explicitly regularized, bound-constrained least-squares problem.

Event-aligned measurement resolution. Boarding and alighting mappings have additional structure that avoids generic repeated graph searches. The strict resolver formerly scanned all links for every measurement to find access links entering a matched departure event or egress links leaving a matched arrival event. It now builds node-keyed CSR indices for those relations once and retains all existing stop, time, trip/line, event-kind, uniqueness, and nonempty-link validations. On the full network, synchronized strict mapping decreased from approximately 121.2 s to 2.31 s: 0.24 s for event/link indices, 1.64 s for 426,436 record resolutions, and 0.42 s for result construction.

All 213,218 alighting measurements map to exactly one egress link. Of 213,218 boarding measurements, 207,805 map to one access link and 5,413 map to two access links because two time-bin access alternatives may enter the same departure event. The compact event-aligned representation therefore stores one primary link per measurement plus only these sparse secondary boarding pairs. It is exact; forcing every row to one link would not be. On the full-network arrays, index storage decreased from 3,454,792 to 1,749,048 bytes and warm aggregation decreased from 0.482 to 0.108 ms, with bit-identical predictions and gradients. This optimization removes mapping startup cost but does not materially change the six-second destination-loading bottleneck.

Explicit fixed-routing adjoint. Generic reverse-mode differentiation through the node-loading scan retains more state than is necessary when link probabilities are fixed. Let c_ℓ be the cotangent of link flow and let a_i be the node-flow adjoint. A reverse topological pass evaluates

$$a_i = \sum_{\ell \in \delta^+(i)} P_\ell \left(c_\ell + a_{h(\ell)} \right).$$

The derivative with respect to initial flow injected at node i is a_i; the existing OD injection operation then gathers this adjoint at origin nodes. The custom vector–Jacobian product therefore retains a node vector and the supplied link cotangent rather than the complete forward scan tape. It deliberately defines no derivative with respect to routing probabilities and is exposed only for the fixed-routing path.

On the representative 465-cell full-network destination, ordinary reverse mode and the explicit adjoint required respectively 12.09 and 6.69 s for a warm value–gradient evaluation. Their resident memory immediately after that evaluation was 16.21 and 13.91 GiB. Objective, prediction, and gradient diagnostics agreed within float32 tolerances. The explicit adjoint is retained as the fixed-routing differentiation path and as a reference for validating future sparse-operator construction.

Stable objective compilation and reuse. The fixed-operator MAP objective is defined at package scope and receives all numerical operator arrays, observations, baselines, and offsets as dynamic JAX arguments. Thus parameter values, observations, and optimizer iteration limits do not alter the compiled signature; shapes, dtypes, backend, JAX/JAXLIB versions, and compilation-relevant configuration still must agree. The public `prepare_ml_objective` and `compile_ml_objective` interface separates tracing, lowering, compilation, executable loading, and first execution, and `run_ml` accepts the resulting executable without creating another jitted closure.

An isolated investigation showed that a previously observed 29.484 s first-call delay was not XLA compilation. Fixed-routing preparation had launched unused asynchronous device work before the valid operator cache was loaded, and the first objective result synchronized that work. The pipeline now validates and loads the cached operator before preparing routing. On the TPG case, isolated tracing takes about 0.026 s, lowering 0.011 s, cold XLA compilation 0.120–0.129 s, executable loading below 0.00001 s, and first execution about 0.0014 s.

The compact persistent artifact remains canonical row-major BCOO storage, with sorted unique indices. Three objective kernels were compared after all scenario preparation had completed. BCOO matvec, direct indexed scatter-add, and sorted segment reduction gave identical float32 objectives and gradients. Their warm value-and-gradient medians were respectively 0.001073 s, 0.000944 s, and 0.000973 s; direct indexed aggregation is therefore used by the stable objective.

Iter.	Cache	Compile/load	Warm V&G	Optimizer	Process	Objective
1	cold	0.1204	0.000975	0.303	5.800	55243.4375
1	populate	0.1285	0.000996	0.292	5.845	55243.4375
1	hit	0.0070	0.000983	0.297	5.167	55243.4375
5	cold	0.1231	0.001001	2.878	8.377	49635.4922
5	populate	0.1194	0.001021	2.604	8.167	49635.4922
5	hit	0.0075	0.000958	2.539	7.622	49635.4922
20	cold	0.1260	0.000991	5.805	11.623	49140.4258
20	populate	0.1260	0.000972	5.777	11.401	49140.4258
20	hit	0.0070	0.000996	5.820	11.047	49140.4258

Table 4: Fresh-process TPG compilation-cache and optimizer measurements. The compile column is cold compilation or persistent executable loading; process time also includes scenario and assignment preparation.

The JAX persistent compilation cache is configured before computation through `PUBLIC_TRANSPORTATION_JAX_COMPILATION_CACHE_DIR` or the public package configuration function. Entry-size and compile-time thresholds default to zero so the objective is eligible. Fresh hit processes create no new cache entry and reproduce cold objectives and gradients. JAX 0.8.1 on the tested Apple CPU emits an AOT target-feature warning while loading entries even though

execution succeeds and results agree exactly; this runtime warning should be rechecked when JAX is upgraded.

Persistent assignment artifacts. Assignment preparation is also demand-independent. Its immutable reusable state consists of the time-expanded graph arrays and labels, destination/OD grouping arrays, and generalized-cost components. It does not contain demand magnitudes, observations, priors, optimizer settings, or compiled executables. The package can store this state as compressed NPZ plus canonical JSON through optional `off`, `auto`, `refresh`, and `readonly` policies; ordinary uncached calls remain the default.

The assignment-cache provenance includes the complete graph-relevant scenario contents, OD keys, assignment and cost configuration, numeric type, package and schema versions, sentinel conventions, and indexing rules. Loaded archives are checked for exact provenance, recognized fields, counts, shapes, dtypes, CSR and padded-mask consistency, and integer/Boolean conventions before JAX arrays are reconstructed. Cache writes use a unique temporary file, synchronization to disk, and atomic replacement. Thus concurrent writers are safe, corrupt or incompatible entries rebuild under `auto`, and `readonly` never writes. The measurement-operator cache retains its own compact-input content hash, so a valid assignment archive cannot validate an incompatible operator.

Internal TPG profiling initially assigned 3.142 s of a 3.313 s preparation to destination grouping. The cause was repeated Python scalar extraction from JAX node arrays, which introduced thousands of small device synchronizations. One NumPy view is now reused for immutable node indexing and validation, and all destination link masks are constructed in one broadcasted operation. Uncached assignment construction consequently fell to 0.097 s: graph construction took about 0.049 s, OD indexing and grouping 0.022 s, and cost construction plus synchronization 0.026 s in the compilation-cache-enabled run.

The TPG artifact contains 36,478 nodes, 11,259 links, 22,022 OD records, and 113 destination groups. Its explicit arrays occupy 6,909,418 bytes and compress to 576,227 bytes. A representative strict cache hit spent 0.00044 s opening the archive, 0.00460 s decompressing arrays, 0.01473 s in combined validation and decompression, 0.000006 s reconstructing host objects, 0.00500 s transferring and synchronizing JAX arrays, and 0.08787 s generating the canonical provenance. The complete assignment-cache call took 0.1084 s.

Policy/case	Assignment	Preparation	Warm V&G	Optimizer	Process
Off	0.0971	1.1756	0.000992	0.2908	2.1939
Populate	0.3412	1.3532	0.000993	0.2975	2.4106
Valid hit	0.1084	1.0775	0.000937	0.2721	2.0211
Corrupt/rebuild	0.3201	1.3339	0.000965	0.3073	2.4081
Mismatch/rebuild	0.3321	1.3463	0.001048	0.3166	2.3935
Read-only hit	0.1128	1.1242	0.000972	0.2922	2.1872

Table 5: Fresh-process TPG assignment-cache measurements in seconds with the measurement-operator and JAX compilation caches enabled.

After eliminating scalar synchronization, uncached reconstruction is already slightly faster than strict cache validation in the assignment stage itself; canonical fingerprinting accounts for most of the hit time. The persistent artifact is therefore useful primarily for strict reproducibility and validation, while the focused grouping simplification provides the main speedup. Relative to the original profile, complete one-iteration time falls from 5.389 s to roughly 2.0–2.2 s. Cache-hit runs at 1, 5, and 20 iterations reproduce objectives 55,243.4375, 49,635.4922, and 49,140.4258 exactly.

3.3.8 Fixed-routing linear MAP estimation at scale

When θ and all generalized costs are fixed, routing probabilities are constant and the complete measurement prediction is affine in the free OD demand. Let $\mathbf{f}_u \in \mathbb{R}^n$ denote the free physical flows, let m be the number of observations, and let $\mathbf{c} \in \mathbb{R}^m$ be the measurement contribution of strictly positive frozen demand. The fixed-routing model is

$$\widehat{\mathbf{y}} = A\mathbf{f}_u + \mathbf{c}, \qquad A \in \mathbb{R}^{m \times n}.$$

Frozen-zero cells have no column in A, while positive frozen cells contribute only to $\mathbf{c}$. Thus neither kind of frozen cell increases the estimation dimension.

The implemented linear MAP problem uses positive observation weights $W = \operatorname{diag}(\boldsymbol{w})$, componentwise physical bounds $\boldsymbol{\ell} \leqslant \mathbf{f}_u \leqslant \mathbf{u}$, and an explicitly selected ordered collection of linear regularization blocks $(L_k, \mathbf{r}_k, \lambda_k)$:

$$\min_{\boldsymbol{\ell} \leqslant \mathbf{f}_u \leqslant \mathbf{u}} \left\{ \frac{1}{2} \left\| W^{1/2}(A\mathbf{f}_u + \mathbf{c} - \mathbf{y}) \right\|_2^2 + \frac{1}{2} \sum_k \lambda_k \left\| L_k \mathbf{f}_u - \mathbf{r}_k \right\|_2^2 \right\}.$$

The supported choices are no regularization, ridge toward the prior demand, and scaled ridge toward the prior demand. No recommendation is applied implicitly. This weighted least-squares mode is distinct from the negative-binomial ML and MAP objectives described below: the two are comparable only when their observation models and priors have deliberately been made equivalent.

Operator representations. All solvers use a common forward–transpose protocol. Small problems may use a dense reference operator. Production alternatives include a native sparse operator with versioned persistence, a matrix-free operator that performs fixed routing during each product, and a sharded sparse operator whose construction and resident storage are bounded. The fixed offset $\mathbf{c}$ is assembled once. Cache identities cover the assignment and OD layouts, fixed demand, measurement mapping, routing parameter, numeric representation, construction configuration, schema, and package version. Cache writes are atomic and an incomplete, corrupt, or incompatible artifact is never accepted as complete.

The monolithic solver interface currently provides a dense reference backend and a bound-constrained TRF/LSMR backend. Results are returned in physical demand units with objective components, residuals, gradients, bound KKT diagnostics, elapsed time, and available forward/transpose product counts. Exact rank and resolution analysis is reserved for small problems. The scalable path instead uses reproducible spectral and randomized probes and reports their convergence and Monte Carlo uncertainty explicitly.

OD blocks without network cuts. For problems too large for a monolithic solve, the package partitions only the free OD columns. The default deterministic order is destination group, time bin, and then free-column index, with deterministic subdivision of oversized groups and optional merging of small compatible groups. Every block uses the complete time-expanded network; partitioning never removes links or prevents a path from traversing another part of the service. User-defined partitions are accepted only if every free column occurs exactly once and no frozen column is present. Block membership and the complete partition are fingerprinted.

Let B be the active block, A_B its independently constructible operator, and $\widehat{\mathbf{y}}$ the current complete prediction. A trial changes only $\mathbf{f}_B$ and is evaluated incrementally as

$$\widehat{\mathbf{y}}_{\text{trial}} = \widehat{\mathbf{y}} + A_B(\mathbf{f}_B^{\text{trial}} - \mathbf{f}_B).$$

The remaining OD variables are held fixed. An update is accepted only if it is finite, respects the bounds, and does not increase the complete objective beyond the numerical acceptance

tolerance. Damping and backtracking are applied when needed. Rejected updates leave the complete state unchanged. Separable quadratic priors are evaluated conditionally; unsupported cross-block prior or constraint coupling is rejected rather than ignored. Exact full products are performed at configurable validation boundaries to detect incremental drift, not after every block.

Anytime state and recovery. Initialization already constitutes a complete feasible approximate solution. After every accepted block or conflict-free parallel batch, the estimator has a complete flow vector, complete prediction, current and best objective, and convergence diagnostics. The durable journal records the replayable flow and prediction deltas using temporary-file creation, synchronization, atomic publication, and a separate commit marker. Periodic compact checkpoints store the complete state and exact next schedule position. Resume is rejected when any authoritative fingerprint changes, including scenario, assignment, OD layout, fixed demand, measurements, prior, routing, partition, solver semantics, or schema. Returned statuses distinguish convergence, time, sweep, and update budgets, graceful interruption with an approximate solution, resource guards, and numerical failure.

Progress events do not claim a generic percentage precision. They distinguish exact, sampled, stale, deferred, and unavailable diagnostics and report objective improvements, projected-gradient measures, block and variable coverage, completed sweeps, elapsed time, checkpoint commitment, and estimated remaining sweep time.

For a bounded large-network pilot, complete-operator products are governed by an explicit fingerprinted policy. The initial prediction may be computed or supplied with validated shape, finiteness, bounds, fixed offset, and provenance. Initial, periodic, and final exact projected gradients can be enabled independently; a disabled diagnostic is recorded as deferred, never relabeled as an exact global quantity. Resume prediction validation may likewise be exact or deferred. Deferred operation avoids an otherwise mandatory global forward or transpose product, but postpones the corresponding global consistency or optimality claim. A separate validation entry point can later recompute the prediction, objective, and projected gradient, including in a fresh process.

A restricted pilot schedule is separately authorized from a complete sweep. Every named block must belong to a completed support group, have compatible persisted exact support, and satisfy support-row, nonzero, operator-storage, and temporary-memory ceilings. The checkpoint records that deterministic schedule and its next position. Numerical semantics and the schedule remain fingerprinted, whereas invocation limits may extend safely, so resuming a five-update run with a six-update budget executes only the sixth update.

The monotonic elapsed-time deadline covers initialization, global products, selected-block construction, solving, and checkpoint persistence. JAX compilation or dispatched device work cannot generally be interrupted inside a process; an indivisible phase can therefore overshoot its allowance. Such an overshoot is reported explicitly, and no subsequent expensive phase is started. Structured diagnostics expose global forward and transpose counts and times, block construction and solve time, checkpoint time, prediction source, and validation status.

The selected-block numerical kernels pass graph and routing arrays as explicit dynamic JAX arguments rather than capturing complete assignment arrays in Python closures. Tracing, lowering, compilation, synchronized device execution, and host transfer are timed separately. A bounded lock-protected compiled-executable cache is keyed by assignment provenance, backend, numeric type, effective OD width, mapped-edge width, graph dimensions, and kernel schema; deadline and callback state are excluded. Deadline checks remain in Python before and after each indivisible JAX stage. On the public example, fresh-process one-variable construction took 0.093–0.194 seconds, with zero captured array constants and identical sparse results; compilation dominated the roughly 0.001-second device execution.

For scheduler diagnostics, the builder can emit structured start and completion events immediately around each phase. An optional append-only JSON-lines sink flushes every complete

record and can call `fsync`; it is separate from operator-cache validity. Consequently, cancellation during XLA compilation leaves a durable `jax_compilation_start` record without publishing an incomplete numerical operator. The public benchmark can stop intentionally after tracing, lowering, compilation, or execution, allowing these indivisible stages to be assigned separate external wall-time limits.

Safe parallelism. Independent block operators can be constructed and cached concurrently. For block solves, the software builds a conflict graph whose nodes are blocks and whose edges represent shared measurement support or declared prior/constraint coupling. A deterministic coloring gives batches with disjoint exact update support. Such blocks may be solved concurrently and their deltas are merged atomically in deterministic order. Overlapping proposals are never treated as independent Gauss–Seidel updates. Construction workers, solver workers, and threads per worker are explicit, and their product is checked against the available CPUs; native numerical thread pools are bounded as well.

Resource preflight and adaptive refinement. The scalable workflow first creates a structural partition and then performs bounded exact-support discovery. The old sharded planner materializes global origin support and retains per-column patterns and future construction tasks; it is therefore a compatibility path for reviewed small cases, not a full-network preflight. The streaming preflight instead prepares and analyzes one destination group at a time. It reduces exact reachable measurement rows immediately to group and block summaries, writes an atomic checkpoint, and releases group-local routing, reachability, pattern, and mapping state before continuing. Retained state is proportional to destination and block summaries, not to OD–measurement support entries.

Four explicit modes distinguish structural inspection, deterministic sampled exact support, complete streaming exact support, and an explicitly authorized materialized compatibility plan. The materialized mode is rejected unless a prior conservative logical-support estimate fits its retained-state budget. Sampled selection includes small, median, 95th-percentile, and maximum structural groups. Its extrapolations report observed ranges and assumptions; sampled completion alone cannot authorize a complete-network pilot.

Elapsed time, process RSS, temporary group state, retained summaries, support rows, nonzeros, and operator bytes are effective limits. Budget exhaustion or interruption produces a typed partial result and a valid atomic checkpoint. Resume validates scenario, assignment, OD layout, fixed demand, measurement mapping, routing, partition, configuration, and schema fingerprints. It neither repeats completed groups nor skips pending groups.

Checkpoint identity separates numerical semantics from invocation policy. The semantic fingerprint covers mode, deterministic sampling and selected groups, support chunking and tolerance, persisted representation, and schema; the authoritative data and routing fingerprints remain mandatory. Elapsed and memory allowances, checkpoint cadence, and deterministic worker settings have a separate recorded policy fingerprint and may change compatibly on resume. Tightened limits are rejected before work if retained summaries or accepted blocks already violate them.

Cumulative support time is never reset and includes discarded work from an incomplete group. In contrast, the elapsed-time allowance applies only to the current invocation and its timer begins at zero after each resume. Reports give previous, current, and cumulative time, invocation count, allowance overshoot, and whether stopping occurred before a group or inside a bounded chunk. Thus exhausting a 120-second allowance does not cause the next 120-second invocation to stop immediately. An unfinished group is not committed and may be recomputed, while every completed group is skipped.

Only after support discovery does the resource preflight select the smallest, median, 95th-percentile, largest, or explicitly named blocks. Before invoking any builder it estimates CSR values and indices, transpose storage, construction temporaries, retained cache, and solver work

arrays. An unsafe block is rejected before allocation; no unselected operator is constructed. For accepted blocks it records machine memory, logical and physical CPUs, cache storage, operator and local-solver memory, construction and product times, and cache/checkpoint sizes. The `auto`, `laptop`, `workstation`, and `server` profiles apply different conservative fractions of available memory. Worker count is bounded by both CPUs and safety-adjusted measured peak memory:

$$n_{\text{workers}} \leqslant \min\left(n_{\text{CPU}}, \left\lfloor \frac{M - M_0}{M_B} \right\rfloor\right),$$

where M is the selected job budget, M_0 is coordinator and assignment memory, and M_B is the conservative measured worker peak. Estimated cache storage is checked separately. The structured recommendation contains hard variable and nonzero ceilings, worker and thread allocation, peak memory, cache size, first-sweep and cache-hit sweep times, and uncertainty. Automatic sizing is never activated until the exact recommendation fingerprint is explicitly accepted.

Selected fixed-routing blocks are constructed directly from their authoritative assignment inputs, compact free-to-active OD layout, destination-group routing, measurement mapping, and persisted exact support. The production builder does not require a complete measurement operator. It combines one or more base OD chunks into a separately configured, memory-guarded OD batch. For each batch, one JIT-compiled forward dynamic program computes node reach probabilities once; fixed-size mapped-edge gathers then aggregate only bounded supported-measurement chunks on the host. Numerical zeros are filtered before canonical COO-to-CSR assembly. Both CSR and CSC storage retain only the union of supported rows, while forward products scatter into the complete measurement coordinate system and transpose products gather from it. Consequently, the forbidden dense array of shape $n_M \times |B|$ is never allocated: the largest dense numerical buffers are bounded by the effective OD batch times the node count, mapped-edge chunk, or supported-measurement chunk.

Measurement-row lookup, enabled-link filtering, global-to-local row translation, and padded mapped-edge plans are invariant for the block and are prepared once, outside the OD-batch loop. The requested batch factor is reduced automatically until conservative reach, mapping, sparse-assembly, routing, solver, and retained storage estimates satisfy both temporary and worker ceilings. Failure of even one base batch is reported before numerical allocation. Because batching does not change the canonical per-column accumulation order, safe batch sizes share one logical cache identity.

Exact selected support and each numerical block are persisted independently by atomic replacement. Their identities include package and schema versions, all scenario, assignment, OD-layout, fixed-demand, measurement, routing, and partition fingerprints, the block and support fingerprints, fixed routing parameter, tolerance, dtype, and canonical accumulation contract. Pure performance chunk sizes are excluded from the logical cache identity. Content hashes, sparse dimensions, row coordinates, and dtype are validated before reuse; corrupt numerical caches are discarded and rebuilt, while corrupt or incompatible support artifacts are rejected. A fresh process can therefore load a block without repeating routing or numerical construction. In-memory retention is explicitly bounded and releasable.

The accepted measurements also define a conservative memory/runtime cost model. Before estimation, any unsafe candidate block is bisected deterministically until every child satisfies the accepted limits. Coverage remains exact, children retain a conservative measurement support, and blocks are never silently enlarged. If a minimum-size block is still unsafe, a resource guard stops the workflow before its operator is constructed. Refinement creates a new partition fingerprint, and the revised schedule is written in the initial checkpoint. A checkpoint for the obsolete parent partition therefore cannot resume the refined run.

This block-coordinate implementation is presently restricted to the fixed-routing linear MAP objective. It must not be used when routing changes with the OD iterate or when the required conditional prior cannot be evaluated exactly.

3.3.9 Opt-out alternative (future extension)

In a general formulation, an opt-out alternative can be introduced for each OD pair and departure interval. In the first implementation, opt-out alternatives are not included: all demand is assumed to be assigned to the public transportation network.

3.4 Reduced-dimensional gravity demand estimation

The public inference layer now provides a minimal dynamic gravity model on the canonical sparse list of feasible OD–departure-time cells. For origin–time production Q_{ot} and fixed positive destination attractiveness A_{dt},

$$q_{odt} = Q_{ot} \frac{I_{odt} A_{dt} \exp\{-\beta_{\text{time}} \widetilde{T}_{odt} - \beta_{\text{transfer}} N_{odt}\}}{\sum_{d'} I_{od't} A_{d't} \exp\{-\beta_{\text{time}} \widetilde{T}_{od't} - \beta_{\text{transfer}} N_{od't}\}}.$$

Masked segmented JAX reductions preserve exact structural zeros and the production totals without materializing a dense origin–destination–time tensor. The time and transfer coefficients, and the negative-binomial dispersion, are mapped from unconstrained optimizer coordinates through stable positive transformations.

Gravity demand is passed directly through the existing dense or BCOO fixed-routing measurement operator, including its frozen-positive measurement offset. The observation layer supports Poisson likelihood and the negative-binomial parameterization $\mathrm{Var}(Y_i) = \mu_i + \mu_i^2/r$. For small parameter counts, a batched-forward derivative applies the routing operator to all demand directions together. The adjoint alternative applies one routing transpose product and a JAX vector–Jacobian product. Neither constructs a dense OD-by-parameter routing Jacobian.

The minimal estimator uses an explicitly traced, lowered, and compiled JAX value-and-gradient kernel with L-BFGS. Its automatic policy benchmarks first and warm executions of the two derivative strategies under a bounded preflight. Atomic checkpoints are written at the initial state and after every completed iteration; their fingerprint covers gravity inputs, model and parameter layouts, routing and measurement provenance, observations, and likelihood semantics. Resume reconstructs the full OD layout, including frozen-positive cells and structural zeros, while restarting non-portable L-BFGS history from the saved iterate. Compilation, first execution, warm execution, strategy selection, persistent-cache configuration, and elapsed optimization time are exposed as structured diagnostics.

A detailed OD table produced this way is not equivalent to estimating every OD cell independently: its detail is induced by the low-dimensional gravity structure and the fixed production and attractiveness inputs.

The first validation mode is deliberately a full-calibration adequacy analysis, not predictive validation. It compares reassigned counts with every observation used for fitting and reports observed and modeled totals, negative-binomial and Poisson deviances, MAE, RMSE, inverse-variance weighted RMSE, standardized negative-binomial residuals, configurable exceedance rates, and observed–modeled quantiles. User-provided measurement-aligned labels produce residual summaries by measurement type, line, direction, stop, time period, origin zone, and destination zone. Ordered vehicle-journey labels enable adjacent standardized-residual correlations. Resulting flags distinguish possible demand restrictions, routing or timing patterns, isolated suspect observations, and systematic overdispersion cautiously: they do not establish causality and never modify the model automatically.

Three atomic relaxations are available when the minimal model is inadequate. For K user-defined groups, each introduces $K-1$ free deviations and derives the final component as the negative sum, so that the full effect vector is exactly centred. The alternatives are destination-zone log-attractiveness deviations, broad time-period deviations multiplying β_{time}, and multiplicative origin-zone corrections to Q_{ot}. A time-period intercept is intentionally excluded because it would cancel within an origin–time softmax. The zone and period mappings are application

inputs and must be contiguous; origin-zone and period labels must also be constant within each origin–time group.

Each child model is warm-started by copying its parent coordinates and setting the new deviations to zero, which reproduces the parent predictions exactly. A configurable ridge penalty on the complete centred effect vector is therefore zero at the parent. Adding or removing one relaxation is explicit and reports its exact parameter count and execution impact. These effects require only indexed cell or group operations and retain the same sparse demand, fixed-routing operator, gradient, checkpoint, and validation machinery.

Candidate relaxations can be screened without fitting them. At the fitted parent, each applicable child is initialized at its exact zero-deviation warm start. With objective gradient g_c and regularized Hessian block H_c in the new child coordinates, the local estimated improvement is

$$\widehat{\mathsf{G}}_c = \tfrac{1}{2}\mathsf{g}_c^\top \mathsf{H}_c^{-1}\mathsf{g}_c.$$

The implementation uses an eigendecomposition and floors non-positive or nearly singular curvature only for this approximation, while reporting the resulting identification warning. The candidate record contains the score statistic, estimated deviance improvement, observation and group support, exact parameter increment, computational cost, recommendation strength, and a plain-language interpretation. It also warns when the parent gradient is too large for an optimum-based diagnostic.

The score is interpreted together with grouped standardized residuals by destination zone, period, and origin zone. Correlations with journey-time and transfer sensitivities can motivate future impedance or transfer-penalty designs, but these are explicitly marked unavailable until a centred child model and its mapping are selected. Coherent within-journey residuals instead warn about possible routing or timing errors, and isolated extremes warn about possible data problems. The catalog is strictly advisory: it neither changes the specification nor estimates or applies a child model.

Completed models are retained in an immutable lineage. Every node has a deterministic identifier and records its parent and applied relaxation, complete specification and parameter layout, raw and physical estimates, feature and compact-layout fingerprints, assignment, graph, and measurement-mapping provenance, optimizer state, assigned-count adequacy diagnostics, runtime and stored-memory diagnostics, checkpoint path, and explicit calibration and validation measurement identities. A child's declared atomic relaxation must remove cleanly to its parent specification. The append-only lineage returns a new value when a child is added, preserving both completed estimation results.

The progression API first loads a completed parent, lists applicable relaxations, and returns the advisory recommendation catalog. It proceeds only after the user passes one explicit relaxation scope. The resulting child layout is initialized from the parent and its predictions are checked against the parent before any optimization. The child is then estimated and validated with the existing checkpointed machinery. The comparison reports changes in objective, likelihood, negative-binomial and Poisson deviance, RMSE, parameter count, and runtime. Calibration and validation measurement identities must remain unchanged during this step. No recommendation is selected automatically, and grouped holdout validation remains a separate subsequent workflow.

After selecting a model structure, predictive validation uses deterministic grouped holdout re-estimation. Complete vehicle journeys, stop-time series, lines, directions, broad time blocks, or explicit application groups are kept indivisible; independent random measurement-row holdout is not supported. A seeded hash orders groups reproducibly. Optional strata may combine measurement type, line, direction, stop, period, origin zone, and destination zone. Groups that cross several stratum values remain intact and use their complete label signature. The resulting complementary calibration and holdout masks, group labels, measurement identity, metadata identity, and policy are serialized under a content fingerprint and can be reused across model structures.

For each selected structure, all parameters are re-estimated using calibration groups only. The resulting complete OD demand is then assigned to the complete measurement operator, and the excluded groups are evaluated as predictions. Calibration and holdout results separately report totals, negative-binomial and reference Poisson deviance, likelihood, MAE, RMSE, and inverse-variance weighted RMSE. The immutable report retains the split and model fingerprints, complete measurement predictions, free and full OD demand, and the fitted result. Tests verify that changing only excluded observations changes holdout metrics but cannot change the estimated parameters.

The recommended workflow is therefore to develop structure with full-data adequacy, select it explicitly, re-estimate under one reusable grouped split, compare predictive evidence, and revise only when that evidence warrants it. The accepted structure may subsequently be refitted on all observations, while the grouped holdout result is retained as the honest predictive assessment.

A public synthetic performance benchmark exercises several gravity parameter counts with dense and BCOO measurement operators. It records tracing, lowering, compilation, first and warm execution, changed-parameter reuse, forward and transpose routing, batched-forward and adjoint value-and-gradient execution, operator storage, process peak memory where measurable, and problem dimensions. The two gradients are checked numerically before timings are accepted. Small-case CPU results show mixed strategy winners with differences often near timer noise; they do not justify raising the conservative eight-parameter guard above which automatic selection avoids a wide batched demand Jacobian. Compiled-handle reuse under changed parameter values is counted directly. Persistent JAX cache hits are reported only by a separate fresh-process protocol because the public in-process API provides no authoritative hit counter.

The complete methodology has also been exercised on the committed public Geneva snapshot, whose OD truth and prior are synthetic. Scheduled path metrics prepare journey times and transfers, while positive prior free demand prepares production and attractiveness offsets. The case contains 96 free cells in a 15,128-cell full layout and 8,967 boarding/alighting measurements. Because a node-by-measurement reverse state is unsuitable at that measurement dimension on the laptop, an exact scalar-column builder compiles the fixed-routing forward map once and constructs BCOO data and indices directly, without a dense operator. The stored operator has 0.556% density and occupies 95,760 bytes.

A bounded two-iteration run completed minimal negative-binomial estimation, full-data adequacy, advisory diagnostics, an exact zero-difference warm start for a selected broad-period child, immutable two-node lineage, and grouped vehicle-journey holdout re-estimation. The holdout contains 864 measurements and the calibration set 8,103. These iteration-limited figures establish public end-to-end software integration; they are not converged estimates and provide no scientific model-selection conclusion.

Two smaller synthetic examples provide worked applications of the same methodology. Simple Example 01 contains four free cells and two frozen-positive cells. Its run reconstructs both fixed cells exactly, exercising the fixed measurement offset; since every free origin–time group contains one destination, this is not evidence for behavioral identification. Simple Example 02 contains 70 free cells, two frozen-positive cells, and 270 measurements. Its sequence estimates the minimal model, constructs an explicitly selected broad-period child with an exact zero-difference warm start, records two-node lineage, and re-estimates the child on 208 calibration measurements before scoring 62 measurements held out by complete vehicle journey. These summaries are instructional checks, not scientific model-selection conclusions.

The gravity objective now consumes a documented operator protocol rather than an explicit matrix. The protocol includes dimensions, fixed offsets, fingerprints, representation, JAX forward, transpose and optional multiple-right-hand-side products, plus declared progress, deadline, cancellation and cache-diagnostic capabilities. Dense and BCOO operators retain their stored matrix for compatibility, while matrix-free implementations satisfy the same contract

without materialization. Thus a large logical measurement-by-OD matrix need never be constructed or transferred to the host. The adjoint value-and-gradient path performs one forward and one transpose product and reuses the routed mean when assembling diagnostics. For a matrix-free operator, automatic derivative selection chooses adjoint directly, avoiding two cold compilations of the complete network; batched-forward remains an explicit small-parameter option. A linear custom-JVP rule also supports forward-mode transformations without materializing the Jacobian. Explicit deadline-governed preparation separates tracing, lowering, compilation, and first and warm execution for forward and transpose products and records the deadline phase, indivisible-operation overshoot, shapes, dtype, backend, and devices. A bounded packaged-example benchmark records these diagnostics, process memory, adjoint objective timings, and checkpoint size while asserting that no global operator was constructed.

Complete fixed-routing preparation itself can be too large when both the effective mask and probability arrays have destination-group-by-link shape. The bounded implementation partitions destination groups canonically, applies both a group-count and a byte ceiling, pads the final shard, and reuses one fixed-shape compiled routing kernel. Each completed shard is flushed and atomically published with strict fingerprints covering the assignment, graph, costs, source masks, destinations, theta, dtype, numerical implementation, schema, package version, and shard plan. An atomic manifest makes deadline or memory stops resumable without recomputing valid shards.

The sharded matrix-free operator loads routing state on demand with bounded LRU residency. Its former Python traversal of every graph node for every group is retained only as a small-case numerical reference. Production forward and reverse products use fixed-shape compiled JAX kernels over all padded groups in a shard, consuming the persisted effective masks and routing probabilities. They neither recompute paths nor routing probabilities. An optional bounded execution batch processes several independent routing shards in one compiled call; a partial final batch is padded without changing canonical OD order. Measurement scatter-add is fused into the compiled forward product, and the reverse kernel is its exact VJP. No complete routing array, link-flow-by-shard stack, Jacobian, or measurement-by-OD matrix is assembled.

The compiled assignment core expresses its destination-group dimension as a sequential `lax.scan`. Consequently, merely enlarging an aggregate batch does not expose independent shard work to additional CPU cores. The multicore implementation instead dispatches already-compiled single-shard scan kernels through a bounded worker pool. Its concurrency is limited by an explicit request, the detected CPU and shard counts, and a ceiling on live in-flight routing bytes. Results are accumulated in canonical shard order, preserving deterministic forward, transpose, and multiple-right-hand-side products. A vectorized-group alternative exposed more cores but was slower on the larger public forward benchmark because it materialized group-by-link intermediates, so it remains experimental.

Structured progress distinguishes product start, batch load, execution, accumulation, completion and deadline-prevented dispatch. Before an indivisible batch, the operator compares remaining absolute-deadline allowance with its recent batch prediction plus a safety margin. Cancellation or an insufficient allowance raises a dedicated interruption exception and discards the partial sum. The estimator propagates its deadline into these products and checkpoints only complete optimizer iterates. On Simple Example 02, four shards covering seven destination groups reused one compiled shape and agreed exactly with complete masks and probabilities; a fresh reload reused all four persisted shards.

After the first measured batch or concurrent wave, deadline admission projects the duration of every remaining wave plus its safety margin. If the complete product is infeasible, a `product_deadline_infeasible` event records completed and total shards, the measured wave, predicted remainder, deadline allowance, discarded partial-work duration, batch size, and concurrency before any further dispatch. Since a partial operator product is not a valid objective evaluation, its accumulated vector is never returned or checkpointed.

A public bounded preflight validates every cache shard and its fingerprints, then, at indi-

vidually selectable safe boundaries, executes one forward product, one reverse product, both three-parameter objective-gradient strategies, runtime projection, and an execution recommendation. It reports the measured warm strategy, current shard batch and residency settings, expected evaluation time and memory, suggested wall time and checkpoint interval, and a laptop-versus-server recommendation. The scalable synthetic benchmark varies graph, link, group, shard, OD, measurement, residency and operator-batch dimensions and reports synchronized forward, reverse and multiple-right-hand-side timing and resource decompositions.

On the 8,192-node public synthetic case, the best aggregate scan required 28.1 ms for a warm forward product at approximately one effective core. Four concurrent scan workers reduced this to 14.7 ms at 2.28 effective cores, and eight workers to 13.9 ms at 2.74 effective cores, a 2.01-fold throughput gain. Reverse and three-column products improved similarly. The benchmark emits a regression warning unless concurrent forward throughput exceeds the aggregate reference by at least ten percent. These results establish the mechanism but do not substitute for a bounded target-cache preflight.

Before the first measured batch is available, the caller may supply a conservative initial batch-duration prediction obtained from preflight. It is used by the same pre-dispatch deadline inequality and is replaced by the first completed empirical duration. This operational estimate does not participate in routing or model fingerprints. For unattended execution, an atomic run manifest records repository and package revisions, numerical fingerprints, dimensions, dtype, theta, operator batching and residency, estimator policy, JAX devices and relevant thread environment. A shared append-only JSONL sink accepts both operator and optimizer progress, timestamps and flushes each record, and optionally synchronizes it to durable storage. Consequently a terminated process leaves both its last complete optimizer checkpoint and a diagnostic record of the active product boundary.

Shard construction may use a bounded thread pool sharing the graph and one XLA executable. Before every dispatch, the parent accounts for current RSS, active-shard estimates, and a safety margin; progress and manifests are committed in canonical order even when execution finishes out of order. A rolling warm- shard estimate prevents launching work that clearly cannot finish before the absolute deadline. Synchronized diagnostics separate input preparation, transfer, kernel dispatch, device synchronization, host transfer, validation, persistence, and cleanup.

Controlled DAG experiments varied groups, nodes, links, outgoing degree, and enabled density. Runtime was essentially unchanged between 10% and 100% enabled density but increased with graph size and group count, confirming full-graph traversal per group. The packaged example is approximately 91% dense, so a compact-support schema is deferred until target-network diagnostics demonstrate sufficient sparsity. Serial, two-worker, and four-worker synthetic preparations produced identical probabilities and compiled one shared executable.

3.5 Implementation notes for fast JAX evaluation

The time-expanded network is fixed across evaluations. All graph connectivity data are precomputed once and stored as immutable arrays: a topological ordering of nodes; compressed representations of outgoing adjacency; and, for each link, tail/head indices and link type.

The backward recursion for the value function and forward propagation for flows are implemented using JAX primitives such as `logsumexp` and scan-like passes over the topological order. Variable out-degrees are handled through CSR-like adjacency and/or padded adjacency with masks, enabling compilation with `jit`. The implementation exposes routing preparation and demand loading as separate JAX-compatible operations. A composed operation preserves the original dynamic assignment behavior, while the fixed-routing inference path supplies prepared probabilities directly to the loading scan.

3.5.1 Deterministic indexing requirement

Reproducibility of inference results requires that the indexing of nodes, links, and OD cells remain strictly deterministic across runs. Graph construction, link ordering, and OD indexing must therefore follow fixed ordering conventions (e.g., sorted iteration and deterministic array construction). Any change in indexing invalidates stored inference outputs, as the Bayesian layer relies on a fixed mapping between parameters and link indices.

4 Measurement Model and Likelihood

Let Y denote observed data (e.g., passenger counts on selected links, or aggregates derived from them). Operational measurements are available only for a subset of quantities. Let $\mathcal{M}$ denote the index set of available measurements. For each $m \in \mathcal{M}$ we denote by Y_m the observed count.

Predicted measurement from the assignment model. The assignment model produces link flows $\mathbf{x}(\mathbf{f};\theta) = \{x_\ell(\mathbf{f};\theta) : \ell \in \mathcal{A}\}$. A deterministic mapping associates each measurement m with a subset of links $\mathcal{A}_m \subseteq \mathcal{A}$, so that the model-predicted quantity is the sum of the corresponding link flows:

$$\lambda_m(\mathbf{f};\theta) = \sum_{\ell \in \mathcal{A}_m} x_\ell(\mathbf{f};\theta).$$

In the current version of the software, aggregation weights are implicitly equal to one.

Asymmetric counting device (undercounting). The counting device is asymmetric: it is more likely to *miss* passengers than to count passengers that are not present. We model this by introducing a detection rate $\rho \in (0,1]$ such that the expected observed count is

$$\mu_m(\mathbf{f};\theta,\rho) = \rho\,\lambda_m(\mathbf{f};\theta).$$

Negative binomial likelihood. Conditionally on $(\mathbf{f},\theta,\rho,r)$, the observations are assumed independent and

$$Y_m \mid \mathbf{f},\theta,\rho,r \;\sim\; \mathrm{NegBin}\big(r,\; p_m(\mathbf{f};\theta,\rho,r)\big), \qquad m \in \mathcal{M},$$

where $r > 0$ is a dispersion parameter and the success probability p_m is defined so that $\mathbb{E}[Y_m] = \mu_m(\mathbf{f};\theta,\rho)$:

$$p_m(\mathbf{f};\theta,\rho,r) = \frac{r}{r+\mu_m(\mathbf{f};\theta,\rho)} = \frac{r}{r+\rho\,\lambda_m(\mathbf{f};\theta)}.$$

Hence, for $y \in \mathbb{N}_0$,

$$\Pr(Y_m = y \mid \mathbf{f},\theta,\rho,r) = \frac{\Gamma(y+r)}{\Gamma(r)\,y!}\,(1-p_m(\mathbf{f};\theta,\rho,r))^y\,(p_m(\mathbf{f};\theta,\rho,r))^r\,.$$

Fixed measurement parameters. In the current version of the software, the detection rate ρ and the dispersion parameter r are treated as fixed, user-specified hyperparameters and are not estimated within the Bayesian inference procedure. Only the OD demand vector and optionally the dispersion parameter θ are inferred. Joint estimation of (ρ, r) is a possible future extension.

Likelihood and normalization. The likelihood is the product over observed measurements:

$$\mathcal{L}\big(\{Y_m\}_{m\in\mathcal{M}} \mid \mathbf{f}, \theta, \rho, r\big) = \prod_{m\in\mathcal{M}} \Pr\big(Y_m = y_m^{\mathrm{obs}} \mid \mathbf{f}, \theta, \rho, r\big)\,.$$

The corresponding log-likelihood is

$$\ell(\mathbf{f}, \theta, \rho, r) = \sum_{m\in\mathcal{M}} \Big[\log\Gamma\big(y_m^{\mathrm{obs}} + r\big) - \log\Gamma(r) - \log\big(y_m^{\mathrm{obs}}!\big) + r\log p_m(\mathbf{f}; \theta, \rho, r) + y_m^{\mathrm{obs}} \log(1 - p_m(\mathbf{f}; \theta, \rho, r))\Big].$$

For posterior computations, terms constant with respect to $(\mathbf{f}, \theta)$ may be dropped.

Alternative observation models (future extension). Other likelihood choices (e.g., Gaussian noise on counts or Poisson models) may be implemented in the future, but the current baseline implementation uses the negative binomial likelihood described above.

5 Free and Frozen OD Demand Cells

An estimation run may hold a selected subset of OD/time-bin cells at predefined nonnegative values. Let $\mathcal{I} = \{1, \ldots, n_{\mathrm{OD}}\}$ be the full assignment indexing, and partition it into the free set $\mathcal{U}$ and the frozen set $\mathcal{F}$, with $\mathcal{U} \cap \mathcal{F} = \varnothing$ and $\mathcal{U} \cup \mathcal{F} = \mathcal{I}$. For each $i \in \mathcal{F}$, let $c_i \geqslant 0$ denote its predefined flow. The demand reconstruction is

$$f_i(z_{\mathcal{U}}) = \begin{cases} f_i^{(0)} \exp(z_i), & i \in \mathcal{U}, \\ c_i, & i \in \mathcal{F}. \end{cases}$$

The smooth bounded transformation described previously is applied to the raw coordinate associated with each z_i, $i \in \mathcal{U}$.

The frozen cells are not represented by dummy parameters. They occur neither in the optimizer vector nor in the Bayesian guide, prior, gradient, Hessian, or covariance matrix. The full vector $\mathbf{f}$ is reconstructed only at the boundary of the assignment model, because assignment necessarily consumes all OD cells. Thus the statistical dimension is exactly

$$d = |\mathcal{U}| + \mathbf{1}\{\theta \text{ is estimated}\},$$

independently of $|\mathcal{F}|$. A frozen value may be zero or positive. In contrast, a free cell requires a strictly positive baseline $f_i^{(0)}$, since the multiplicative parameterization cannot move away from a zero baseline.

Frozen values are supplied by a sparse CSV file with columns `origin_stop_id`, `dest_stop_id`, `time_bin_id`, and optionally `fixed_flow`. A missing or blank `fixed_flow` denotes zero. Keys are validated against the scenario, duplicates are rejected, and the free/frozen partition is constructed in the canonical assignment order. The same immutable partition is passed to ML, MAP, and Bayesian VI. Saved results include the free indices, frozen indices, and frozen values so that downstream processing can verify the invariant. The layout also has its own canonical SHA-256 fingerprint, computed from the OD keys, partition indices, free baselines, and frozen values. This fingerprint is stored separately from the network/assignment fingerprint: changing only the frozen-demand file must therefore change the layout identity even though the network identity remains unchanged. The canonical JSON payload used to compute the digest is persisted with the result. On loading, the digest is recomputed from that payload to detect corruption or manual modification. During post-processing, the layout is rebuilt from the current scenario and `fixed_demand.csv`; its fingerprint must match the result fingerprint before assignment or report generation proceeds. The network fingerprint and layout fingerprint are checked independently.

5.1 Topology-driven structural-zero preprocessing

The fixed-demand file may be generated by a deterministic preprocessing service before ML, MAP, or Bayesian estimation. Its single substantive input is a versioned TOML configuration. The configuration identifies the scenario and output folders, selects rules in a dedicated `rules.enabled` table, and provides a separate table for the parameters of each enabled rule. Unknown keys, missing required keys, invalid types, invalid ranges, and missing input paths are errors. Relative paths are resolved relative to the TOML file.

The service builds the same production time-expanded topology used for assignment, with the configured access-deviation, transfer-wait, and minimum dwell assumptions. A backward dynamic program is performed once per destination and evaluates only OD/time keys already present in the demand candidate table. It records path feasibility, minimum transfers, minimum initial wait, minimum journey time, number of feasible departures, and earliest arrival. Boarding the first vehicle is not a transfer, and arrival at the destination is absorbing.

Two core rules classify a cell as structural zero when no feasible path exists, or when the minimum feasible transfer count exceeds the accepted maximum. With `max_transfers = 2`, paths requiring three transfers or more are therefore excluded. Optional rules cover identical origin and destination stops, excessive initial wait, excessive journey time, and too few feasible departures. Equality at a configured maximum or minimum is retained. All triggered reasons are saved, while a fixed precedence supplies one primary reason for mutually exclusive summary counts.

An existing fixed-demand file can be reconciled with the detected zeros. The only implemented conflict policy is *error*: a nonzero existing value on a newly detected structural zero stops preprocessing. Compatible zero and positive fixed values are preserved exactly, and newly detected cells are added with `fixed_flow = 0`. Duplicate or unknown keys, negative or non-finite values, and all conflicts are reported without producing a partial merged result.

The output transaction writes a canonically sorted `fixed_demand.csv`, a per-cell audit CSV, a JSON summary, the fully resolved TOML configuration, and scenario, graph, and configuration fingerprints with content hashes. Every file is rendered and validated before atomic replacement. Passing the generated fixed-demand file to estimation removes those structural-zero cells from the parameter vector altogether; they are not zero-valued dummy parameters.

Potentially long phases expose an optional dependency-free progress callback. Immutable events report a stable phase name, completed and total work, elapsed time, and an optional artifact message. The destination-profile, cell-classification, and audit-rendering loops report every item for small problems and use count- or ten-second throttling for large problems, always including zero and the exact final total. A context-managed optional `tqdm` adapter displays one phase at a time on standard error, leaving standard output suitable for machine-readable JSON. Callback exceptions propagate normally, and rendering failures occur before atomic file replacement, so progress reporting does not permit a partial file to replace a valid artifact.

If $|\mathcal{U}| = 0$ and θ is fixed, then $\mathrm{d} = 0$. This is treated as a deterministic evaluation rather than as an optimization or variational-inference problem. The likelihood is evaluated once for validation, while NumPyro SVI and the numerical optimizer are bypassed entirely. Returned parameter, gradient, Hessian, covariance, and posterior-coordinate arrays are correspondingly empty.

Complexity boundary. The software exposes a structural complexity profile so that dimensionality can be audited without relying on machine-dependent timings. For reduced dimension

d and effective low rank r_q^{eff}, the principal statistical state sizes are

state	number of scalar entries
optimizer vector or gradient	d
diagonal Gaussian guide	$2d$
low-rank Gaussian guide	$2d + dr_q^{\text{eff}}$
full Gaussian guide	$d + d(d+1)/2$
dense Hessian or covariance	d^2.

These statistical quantities depend only on the free cells and optional θ, not on the number of frozen cells.

There is a separate deterministic cost boundary. Each likelihood evaluation passes a compact demand vector containing the n_{free} free cells and the $n_{\text{fixed},+}$ strictly positive frozen cells. Frozen-zero cells are absent from that vector, from the padded OD group arrays, and from assignment demand injection. A destination group is also removed when it contains no remaining cell. Thus the assignment demand-vector size is $n_{\text{free}} + n_{\text{fixed},+}$ rather than n_{OD}. The time-expanded graph and other scenario-wide static arrays remain unchanged, so forward runtime still depends on the graph and on the surviving destination groups. Positive frozen flows must participate in assignment predictions; when θ is estimated, even their route-choice probabilities change with the estimated parameter. The full n_{OD} vector is reconstructed only after estimation for persistence and reporting. For every VI, ML, or MAP problem, the implementation reports the total, free, frozen-zero, and frozen-positive OD counts; the compact assignment-vector size; and the original, surviving, and removed destination-group counts. These structural metrics accompany elapsed time so that runtime differences can be interpreted in terms of the work actually performed by the assignment model.

6 Maximum Likelihood and Maximum A Posteriori Estimation

The differentiable assignment and measurement models can be used for point estimation without constructing a full posterior approximation. Let

$$\eta = \begin{cases} (\tilde{z}_u, \tilde{u}), & \text{if } \theta \text{ is estimated,} \\ \tilde{z}_u, & \text{if } \theta \text{ is fixed,} \end{cases}$$

denote the vector of raw unconstrained parameters. The same smooth transformations defined above map η to the effective demand vector f and, when applicable, to θ. Consequently, the likelihood-based ML, MAP, and Bayesian methods use the same forward model and negative binomial likelihood. The separate fixed-routing linear mode uses the weighted objective defined in Section 3.3.8.

6.1 Maximum likelihood estimation

The maximum likelihood estimator (MLE) is

$$\hat{\eta}_{\text{ML}} = \arg\max_{\eta} \ell(\eta),$$

where

$$\ell(\eta) = \log \mathcal{L}(Y \mid f(\tilde{z}), \theta(\tilde{u}), \rho, r)\,,$$

with the dependence on $\tilde{u}$ omitted when θ is fixed. The MLE uses no prior information. The baseline demand $f^{(0)}$ still appears in the parameterization $f = f^{(0)} \exp(z)$, but it is not a probabilistic prior in the ML objective.

After optimization, the raw point estimate is transformed using exactly the same smooth mappings as in Bayesian inference:

$$\widehat{z}_q^t = z_{\max} \tanh(\widehat{\widetilde{z}}_q^t / z_{\max}), \qquad \widehat{f}_q^t = f_q^{t,(0)} \exp(\widehat{z}_q^t),$$

and, when estimated,

$$\widehat{u} = u_{\max} \tanh(\widehat{\widetilde{u}} / u_{\max}), \qquad \widehat{\theta} = \exp(\widehat{u}).$$

6.2 Maximum a posteriori estimation

Maximum a posteriori (MAP) estimation uses the same likelihood and priors as the Bayesian model, but returns only the posterior mode:

$$\widehat{\eta}_{\mathrm{MAP}} = \arg\max_{\eta} \{\ell(\eta) + \log p(\eta)\}.$$

Here $p(\eta)$ contains the Gaussian prior on $\widetilde{z}$ and, when θ is estimated, the Gaussian prior on $\widetilde{u}$. The priors are evaluated on the raw variables, not on their smoothly bounded transforms.

The software implements ML and MAP through the common penalized objective

$$Q_w(\eta) = -\ell(\eta) - w \log p(\eta), \qquad w \geqslant 0.$$

The setting $w = 0$ gives pure maximum likelihood and $w = 1$ gives MAP under the same prior as the Bayesian model. Intermediate or larger values provide regularized point estimates, but do not in general correspond to the posterior of the stated Bayesian model.

6.3 Numerical optimization and local uncertainty

The current point-estimation implementation minimizes Q_w with gradient-based SciPy optimizers such as BFGS or L-BFGS-B. Gradients are supplied by JAX automatic differentiation. Because the objective need not be globally convex, convergence to the global optimum is not guaranteed; initialization, stopping tolerances, and local curvature should therefore be inspected.

When requested, the software computes the Hessian of Q_w at the optimum and uses its inverse as a local covariance approximation. For ML this is an asymptotic likelihood-based approximation; for MAP it is a Laplace-style local approximation to posterior curvature. These standard errors are not equivalent to a complete posterior distribution and may be unreliable for weakly identified, asymmetric, bounded, or multimodal problems.

7 Bayesian Inference via Variational Inference

Let $\widetilde{z}_U$ denote the vector of raw OD-deviation variables for free cells and let $\widetilde{u}$ denote the raw variable associated with the dispersion parameter of the route choice model. The effective parameters supplied to the forward model are deterministic transformations of these raw variables. The posterior distribution is proportional to

$$p(\widetilde{z}_U, \widetilde{u} \mid Y) \propto \mathcal{L}(Y \mid f(\widetilde{z}_U), \theta(\widetilde{u}))\ p(\widetilde{z}_U)\, p(\widetilde{u}).$$

When θ is fixed, $\widetilde{u}$ and its prior are omitted.

Exact Bayesian inference is computationally intractable for high-dimensional OD vectors. The current implementation therefore uses *stochastic variational inference* (SVI) as implemented in NumPyro.

Variational approximation. The posterior is approximated by a parametric distribution $q_\phi(\tilde{z}_\mathcal{U}, \tilde{u})$ in the raw unconstrained parameter space. The parameters ϕ are chosen to maximize the *Evidence Lower Bound* (ELBO):

$$\mathrm{ELBO}(\phi) = \mathbb{E}_{q_\phi}\left[\log p(Y, \tilde{z}_\mathcal{U}, \tilde{u}) - \log q_\phi(\tilde{z}_\mathcal{U}, \tilde{u})\right].$$

Maximizing the ELBO is equivalent to minimizing $\mathrm{KL}(q_\phi \,\|\, p)$ between the variational approximation and the true posterior.

Guide family. The baseline implementation uses Gaussian variational families in an unconstrained parameter space (e.g., diagonal or low-rank approximations), and optimization is performed using stochastic gradients (Adam). For a requested low-rank value r_q, the implemented effective rank is

$$r_q^{\mathrm{eff}} = \min(r_q, d),$$

where $d = |\mathcal{U}| + \mathbf{1}\{\theta \text{ is estimated}\}$ is the reduced statistical dimension. This prevents a guide configured for the original model from retaining unnecessary covariance factors after many OD cells are frozen. The effective rank, rather than the oversized requested value, is recorded in the inference diagnostics. When $d = 0$, no guide is constructed.

Prior on θ. The dispersion parameter θ is strictly positive. When it is estimated, the implementation introduces an unconstrained raw variable $\tilde{u} \in \mathbb{R}$ and defines

$$u = u_{\max} \tanh\left(\frac{\tilde{u}}{u_{\max}}\right), \qquad \theta = \exp(u), \qquad u_{\max} > 0.$$

Thus θ is positive and smoothly bounded between $\exp(-u_{\max})$ and $\exp(u_{\max})$. The configuration currently exposes $u_{\max}$ under the historical name `u_clip`, although no hard clipping is performed.

A Gaussian prior is placed on the raw variable,

$$\tilde{u} \sim \mathcal{N}(\mu_{\tilde{u}}, \sigma_u^2).$$

If the requested center in effective log-dispersion space is μ_u—for example, $\mu_u = \log(\theta^{(0)})$ for a baseline value $\theta^{(0)}$—the raw prior center is chosen by the inverse transformation,

$$\mu_{\tilde{u}} = u_{\max} \operatorname{arctanh}\left(\frac{\mu_u}{u_{\max}}\right).$$

This guarantees that transforming the raw prior center gives exactly the requested effective center. It requires $|\mu_u| < u_{\max}$.

Posterior transformation and reporting. Posterior samples produced by the variational guide are samples of the raw variables $\tilde{z}$ and, when applicable, $\tilde{u}$. Before reporting demand or θ, the software applies the same smooth transformations used during likelihood evaluation:

$$z_q^t = z_{\max} \tanh(\tilde{z}_q^t / z_{\max}), \qquad f_q^t = f_q^{t,(0)} \exp(z_q^t),$$

and

$$u = u_{\max} \tanh(\tilde{u}/u_{\max}), \qquad \theta = \exp(u).$$

Consequently, reported posterior summaries correspond exactly to the parameter values used by the assignment and likelihood, rather than to untransformed raw samples.

Alternative inference engines. Markov chain Monte Carlo methods could be used in principle, but they are computationally expensive in high-dimensional OD settings. Support for alternative probabilistic programming frameworks (e.g., PyMC) may be considered in the future; the present baseline is NumPyro+SVI.

8 Comparison of the Estimation Methods

The three estimation approaches share the assignment model, measurement equation, likelihood, raw parameter space, and smooth parameter transformations. They differ in the statistical quantity they target.

ML versus MAP. The MLE maximizes the likelihood alone, whereas MAP balances the likelihood with the specified prior. They therefore need not coincide. MAP is typically pulled toward the prior center and can be more stable in weakly identified problems. As the amount of informative data increases and the likelihood dominates the prior, the ML and MAP estimates will often become close, subject to regularity and identifiability conditions.

MAP versus Bayesian inference. If MAP and Bayesian inference use exactly the same likelihood, priors, and parameterization, the MAP estimate is the mode of the exact Bayesian posterior. It is not generally equal to the posterior mean or median. Equality between the mode and mean occurs only in special cases, such as an exactly symmetric unimodal posterior. The difference may be substantial for skewed, weakly identified, bounded, or multimodal posteriors.

Variational approximation. The implemented Bayesian procedure reports means and other summaries computed from the variational approximation q_{ϕ}, not from the exact posterior. Differences between MAP and VI results can therefore arise from three distinct sources:

1. the intrinsic difference between a posterior mode and a posterior mean;
2. approximation error introduced by the selected variational guide; and
3. optimization and finite posterior-sampling error in the VI procedure.

The posterior draws used to compute VI summaries introduce Monte Carlo variability, but this is not the only, and often not the dominant, reason for differences between the estimates.

When estimates should be close. Under standard regularity conditions, with a large informative sample and a well-identified model, the posterior is often approximately Gaussian and concentrated. In that regime the MLE, MAP estimate, posterior mode, and posterior mean may be close. This is an asymptotic property rather than an equality guaranteed by the software.

Recommended comparisons. For validating the implementation, pure ML should be compared across optimizers using $w = 0$, and MAP should be compared with the mode of the same posterior using $w = 1$. Comparing an ML or MAP point estimate directly with the reported VI posterior mean is useful diagnostically, but disagreement should not automatically be interpreted as a software error. In every comparison, the effective transformed values $(\mathbf{f}, \theta)$ should be compared rather than the raw variables $(\widetilde{\mathbf{z}}, \widetilde{\mathbf{u}})$.

9 Discussion

This software framework combines timetable information and operational counts to infer OD demand and quantify uncertainty, while using a differentiable assignment model compatible with ML, MAP, and gradient-based Bayesian inference. ML and MAP provide computationally convenient point estimates and local curvature diagnostics, whereas Bayesian VI additionally provides an approximate distribution that represents parameter uncertainty.

Planned extensions of the software include incorporating capacity effects in a differentiable manner, estimating additional measurement parameters (e.g., ρ and $\mathtt{r}$), supporting weighted measurement aggregations, and adding opt-out alternatives.

References

Dial, R. B. (1971). A probabilistic multipath traffic assignment model which obviates path enumeration, *Transportation research* **5**(2): 83–111.